

6.3 Bayes Theorem and Concept Learning – Teacher Notes

1. Relationship of Bayes Theorem to Concept Learning

Bayes theorem lets us calculate the **posterior probability** of each hypothesis given training data.

In principle: - Compute $P(h \mid D)$ for **every hypothesis** $h \in H$. - Output the most probable hypothesis (MAP).

This defines a **brute-force Bayesian concept learning algorithm**.

Later: Compare this with algorithms from Chapter 2 (like FIND-S, Candidate-Elimination) → under certain conditions they yield the same result as brute-force Bayes.

2. Brute-Force MAP Learning Algorithm

Steps: 1. For each hypothesis $h \in H$, calculate the posterior probability $P(h \mid D)$. 2. Output the hypothesis h_{MAP} with the highest posterior probability.



Problem: For large hypothesis spaces, this brute-force approach is computationally expensive.

Importance: Provides a **gold standard** against which other efficient algorithms can be judged.

3. Assumptions for Simplification

We choose simple priors and likelihoods:

- Training data is **noise-free** → $d_i = c(x_i)$.
- The **true concept is in H**.
- All hypotheses are **equally probable a priori**.

Therefore:

- **Prior probability:**

$$P(h) = \frac{1}{|H|}, \quad \forall h \in H$$

- **Likelihood:**

$$P(D | h) = \begin{cases} 1, & \text{if } h \text{ is consistent with all training examples} \\ 0, & \text{otherwise} \end{cases}$$

4. Deriving Posterior Probability

Step 1: Recall Bayes theorem

$$P(h | D) = \frac{P(D | h)P(h)}{P(D)}$$

- We set: $P(h) = 1/|H|$ (uniform prior). - $P(D | h) = 1$ if h is consistent with all examples, else 0.

Step 2: Case when h is consistent with D - Numerator = $P(D | h)P(h) = 1 \cdot (1/|H|) = 1/|H|$. - Denominator = $P(D)$. - So:

$$P(h | D) = \frac{1/|H|}{P(D)}$$

Step 3: Compute $P(D)$

$$P(D) = \sum_{h \in H} P(D | h)P(h)$$

- Only consistent hypotheses contribute. - If there are $|VS_{H,D}|$ consistent hypotheses, each contributes $1/|H|$. - So:

$$P(D) = |VS_{H,D}| \cdot (1/|H|)$$

Step 4: Put it together

$$P(h | D) = \frac{1/|H|}{|VS_{H,D}|/|H|} = 1/|VS_{H,D}|$$

5. Meaning

- If a hypothesis is **inconsistent** $\rightarrow P(h | D) = 0$.
- If a hypothesis is **consistent** $\rightarrow P(h | D) = 1/|VS_{H,D}|$.
- Thus, the posterior probability is shared **equally** among all consistent hypotheses in the version space.

 Interpretation: If h is consistent with the training data, its posterior probability is simply the reciprocal of the size of the version space.

6. Diagram: Evolution of Probabilities

The following figure shows how probabilities change as more data arrives:

- **(a)** Initial prior: $P(h)$, all hypotheses equally likely.
- **(b)** After first example: $P(h \mid D_1)$, inconsistent hypotheses eliminated.
- **(c)** After more examples: $P(h \mid D_1, D_2)$, only one hypothesis remains with probability 1.

Posterior Probabilities Evolution

7. Conclusion from Analysis

The above analysis implies that under our choice for $P(h)$ and $P(D \mid h)$: - Every consistent hypothesis has posterior probability $1/|VS_{H,D}|$. - Every inconsistent hypothesis has posterior probability 0. - Therefore, **every consistent hypothesis is a MAP hypothesis**.

6.3.2 MAP Hypotheses and Consistent Learners

The above analysis shows that in the given setting, every hypothesis consistent with D is a MAP hypothesis. This translates directly into an important observation: a broad class of learners known as **consistent learners**.

Consistent Learners

- A learning algorithm is a consistent learner if it outputs a hypothesis with **zero training error**.
- Under uniform prior probabilities over H and deterministic, noise-free training data: every consistent learner outputs a **MAP hypothesis**.

FIND-S Algorithm (Example)

- FIND-S searches from specific to general hypotheses, outputting a maximally specific consistent hypothesis (a member of the version space).
- Because FIND-S outputs a consistent hypothesis, it is guaranteed to be a MAP hypothesis under the distributions defined above.
- Though FIND-S does not explicitly manipulate probabilities, we can characterize its behavior by specifying distributions for $P(h)$ and $P(D \mid h)$ under which its output is MAP.

Other Priors

- Suppose $P(h)$ is any prior distribution that favors more specific hypotheses (i.e., $P(h_1) \geq P(h_2)$ if h_1 is more specific than h_2).
- Then FIND-S still outputs a MAP hypothesis under this prior and the same likelihood assumptions.

Bayesian Characterization of Learning Algorithms

- The Bayesian framework provides one way to characterize algorithms like FIND-S or Candidate-Elimination, even though they don't explicitly use probabilities.
- By identifying priors and likelihoods under which their outputs are MAP, we can describe their **implicit assumptions**.

- This parallels the concept of **inductive bias**: the assumptions sufficient to justify a learner's inferences.
- Example: Candidate-Elimination assumes the target concept is in H .
- Bayesian perspective reframes this as:
- Priors $P(h)$ specify preference over hypotheses.
- Likelihoods $P(D | h)$ specify how data supports/rejects hypotheses.

Special Case

- Here $P(D | h) \in \{0, 1\}$ due to noise-free assumption.
 - Later sections extend this to **noisy data**, where $P(D | h)$ can take intermediate values.
-